Cell 10: RDF Triple Generation

Summary

Purpose

The purpose of **Cell 10** is to **clean, validate, standardize, and deduplicate the RDF triples** produced in Cell 9 before they are used to build the Knowledge Graph.

Although Cell 9 generated RDF triples, they may contain:

- Duplicate triples
- Missing values
- Self-loops (document linking to itself)

Cell 10 removes these issues and prepares a **final RDF table**.

Input from Cell 9

Subject Predicate Object

OpenTCCM uses MATLAB

OpenTCCM uses MATLAB

Engine depends_on Fuel System

Rule detailed_in Test Document

Engine NULL Fuel System

Notice:

- Duplicate row
- Missing predicate

Output of Cell 10

Subject Predicate Object

OpenTCCM uses MATLAB

Engine depends_on Fuel System

Rule detailed_in Test Document

Now every row is a **clean, unique RDF triple**.

This cleaned RDF table will be used to build the Knowledge Graph.

Code Explanation

Stage 1: Filter and Deduplicate Semantic Records

```
validated_rdf_df = (
   rdf_df
   ...
)
```

Purpose

Take the RDF triples from Cell 9 and clean them.

Input

rdf_df

Output

validated_rdf_df

Step 1: Remove Incomplete RDF Triples

```
.filter(
   F.col("subject_id").isNotNull() &
   ...
)
```

Purpose

Keep only complete RDF triples.

A valid RDF triple must have:

- Subject

- Predicate
- Object

Example

Before

Subject	Predicate	Object
Engine	depends_on	Fuel
Engine	NULL	Fuel
NULL	uses	MATLAB

After

Subject	Predicate	Object
Engine	depends_on	Fuel

Step 2: Remove Self-Loops

```
.filter(
    F.col("subject_id") != F.col("object_id")
)
```

Purpose

Remove cases where a document links to itself.

Example

Invalid

Engine

|

depends_on

|

Engine

Subject = Object

This relationship is not useful.

It is removed.

Step 3: Remove Duplicate RDF Triples

.dropDuplicates([

"subject_id",

"predicate",

"object_id"

])

Suppose

(OpenTCCM, uses, MATLAB)

(OpenTCCM, uses, MATLAB)

(OpenTCCM, uses, MATLAB)

Only one triple is kept.

Result

(OpenTCCM, uses, MATLAB)

Step 4: Standardize Column Names

.select(...)

Purpose

Rename columns into the final RDF format.

Old

subject_id

object_id

New

subject

object

Also keep

- subject_title
- object_title
- sentence
- link_type

Final table

subject predicate object

Stage 2: Predicate Distribution

predicate_distribution

Purpose

Count how many times each relationship appears.

Example

Suppose RDF table contains

uses

uses

uses

depends_on

depends_on

part_of

Output

Predicate	Count
uses	3
depends_on	2
part_of	1

This helps understand

- Most common relationship
- Graph statistics
- Data quality

Stage 3: RDF Summary

input_count = rdf_df.count()

output_count = validated_rdf_df.count()

Purpose

Compare

Before cleaning

↓

After cleaning

Example

Input RDF Records : 5500

Output RDF Triples : 4800

Meaning

700 invalid or duplicate records were removed.

Print Predicate Distribution

for row in predicate_distribution.collect():

Example

uses : 320

depends_on : 250

part_of : 180

owned_by : 70

Display Sample RDF Triples

validated_rdf_df.limit(10)

Purpose

Verify the output.

Example

Subject

Engine

Predicate

depends_on

Object

Fuel System

Display

(Engine, depends_on, Fuel System)

Display Final RDF Table

display(validated_rdf_df)

This is the final RDF table that will be used for graph construction.

Cell 10 Workflow

Cell 9 Output

rdf_df

|

▼

Remove Incomplete RDF Triples

|

▼

Remove Self-Loops

(Subject ≠ Object)

|

▼

Remove Duplicate Triples

|

▼

Rename & Standardize Columns

|

▼

validated_rdf_df

|

▼

Calculate Predicate Distribution

|

▼

Display RDF Summary

|

▼

Pass to Graph Construction

Example End-to-End

Cell 9 Output

Subject	Predicate	Object
OpenTCCM	uses	MATLAB
OpenTCCM	uses	MATLAB
Engine	depends_on	Fuel
Engine	NULL	Fuel
Rule	part_of	Rule

↓

Remove NULL Values

Subject	Predicate	Object
OpenTCCM	uses	MATLAB
OpenTCCM	uses	MATLAB
Rule	part_of	Rule

↓

Remove Self-Loops

(Removes Rule → part_of → Rule)

Subject	Predicate	Object
OpenTCCM	uses	MATLAB

Subject	Predicate	Object
OpenTCCM	uses	MATLAB

↓

Remove Duplicates

Subject	Predicate	Object
OpenTCCM	uses	MATLAB

↓

Final RDF Table

Subject	Predicate	Object
OpenTCCM	uses	MATLAB

Final Output of Cell 10

The final output is **validated_rdf_df**, which contains **clean, unique, and valid RDF triples**.

Each row has:

- **Subject** → Source XML reference (node)
- **Predicate** → Semantic relationship extracted by the LLM
- **Object** → Target XML reference (node)

This DataFrame is now ready to be used in the **next cell**, where the actual **Knowledge Graph (NetworkX graph) is constructed by creating nodes and edges from these validated RDF triples**.

Cell 9 vs Cell 10

Cell	Purpose	Output
Cell 9	Extract semantic relationships using the LLM	Raw RDF triples (rdf_df)
Cell 10	Clean, validate, deduplicate, and standardize those triples	Final RDF table (validated_rdf_df) ready for graph construction

Stage 9: Knowledge Graph Construction

Cell 11 (Cell 13 in your notebook): Build Knowledge Graph from RDF Triples

---

Summary

Purpose

The purpose of this cell is to **convert the validated RDF triples into a directed Knowledge Graph using NetworkX**.

Until Cell 10, the relationships existed only as rows in the validated_rdf_df DataFrame.

Example:

Subject	Predicate	Object
Engine	depends_on	Fuel System
Fuel System	contains	Pump
OpenTCCM	uses	MATLAB

These are just records.

Cell 11 transforms these records into an actual graph.

Input

The input is the cleaned RDF DataFrame:

validated_rdf_df

Example:

Subject	Predicate	Object
Engine	depends_on	Fuel System
Fuel System	contains	Pump

Output

A NetworkX directed graph:

Engine

|

depends_on

▼

Fuel System

|

contains

▼

Pump

Now every XML document becomes a **graph node**, and every RDF triple becomes a **graph edge**.

Code Explanation

Step 1: Import NetworkX

import networkx as nx

Purpose

Import the NetworkX library.

NetworkX provides graph data structures and algorithms.

Without NetworkX

RDF Triples

With NetworkX

Knowledge Graph

Step 2: Create Directed Graph

KG = nx.DiGraph()

Purpose

Create an empty directed graph.

Initially

KG

Nodes : 0

Edges : 0

Why Directed?

Because relationships have direction.

Example

Engine ----depends_on----> Fuel System

This is **not** the same as

Fuel System ----depends_on----> Engine

So a directed graph (DiGraph) is the correct choice.

Step 3: Collect RDF Triples

validated_triples = validated_rdf_df.collect()

Purpose

validated_rdf_df is a **Spark DataFrame** distributed across the Databricks cluster.

NetworkX runs only on the driver (single machine), so you must bring the RDF triples back to the driver.

After collect(), you have a Python list of rows.

Example:

[

(Engine, depends_on, Fuel System),

(Fuel System, contains, Pump),

(OpenTCCM, uses, MATLAB)

]

Step 4: Process Each RDF Triple

for row in validated_triples:

The loop processes **one RDF triple at a time**.

Example

First iteration

Subject

Engine

Predicate

depends_on

Object

Fuel System

Second iteration

Fuel System

contains

Pump

Step 5: Read Triple Information

subject_ref = row["subject"]

object_ref = row["object"]

subject_title = row["subject_title"]

object_title = row["object_title"]

Example

subject_ref

doc:Engine

subject_title

Engine

object_ref

doc:Fuel_System

object_title

Fuel System

Notice

The graph key is the XML reference.

The title is stored as metadata.

Step 6: Add Subject Node

KG.add_node(

  subject_ref,

  title=subject_title

)

Suppose

doc:Engine

doesn't exist.

NetworkX creates

Node

doc:Engine

Attributes

title = Engine

If the node already exists

Nothing happens.

NetworkX ignores duplicates.

Step 7: Add Object Node

```
KG.add_node(
   object_ref,
   title=object_title
)
```

Similarly

doc:Fuel_System

becomes another node.

Now

doc:Engine

doc:Fuel_System

exist in the graph.

Step 8: Add Semantic Edge

```
KG.add_edge(
    subject_ref,
    object_ref,
    predicate=row["predicate"],
    sentence=row["sentence"],
    link_type=row["link_type"]
)
```

This is the most important line.

It connects two XML documents.

Suppose

Subject

Engine

Predicate

depends_on

Object

Fuel System

The graph becomes

Engine

|

depends_on

▼

Fuel System

The edge also stores metadata:

- predicate
- sentence
- link_type

Example edge attributes:

predicate : depends_on

sentence : Engine depends on Fuel System for fuel delivery.

link_type : wiki

This metadata is very useful during GraphRAG retrieval because you can retrieve not only the connected node but also the original sentence that explains the relationship.

Graph After Multiple RDF Triples

Suppose the RDF table contains:

Subject	Predicate	Object
Engine	depends_on	Fuel System
Fuel System	contains	Pump
Pump	connected_to	Valve

The graph becomes:

Engine

|

depends_on

▼

Fuel System

|

contains

▼

Pump

|

connected_to

▼

Valve

Now all XML documents are interconnected.

Step 9: Graph Summary

KG.number_of_nodes()

Returns

Total Nodes : 20458

Meaning

20,458 XML documents became graph nodes.

KG.number_of_edges()

Returns

Total Edges : 78120

Meaning

78,120 semantic relationships exist.

Step 10: Display Sample Nodes

sample_nodes

Example

Node

doc:Engine

Title

Engine

This verifies that nodes were created correctly.

Step 11: Display Sample Edges

Example

Display

Engine

depends_on

Fuel System

Internally

(doc:Engine)

↓

(doc:Fuel_System)

This confirms that the graph stores both the XML reference (used as the unique node key) and the human-readable title (used for display).

Overall Workflow

Cell 10 Output

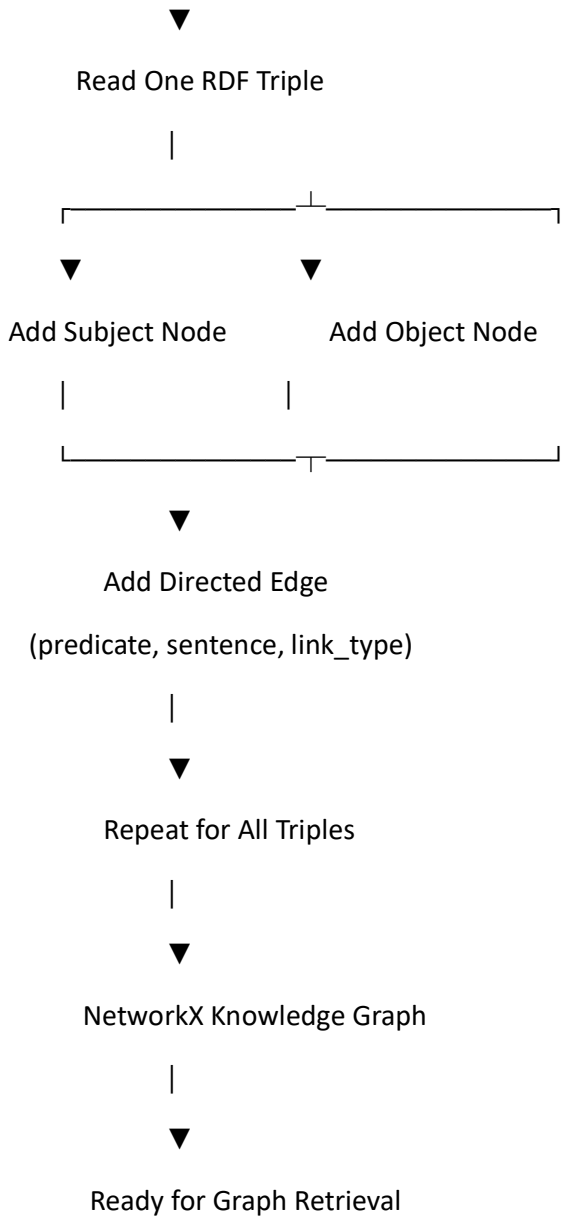
validated_rdf_df

|

▼

Collect RDF Triples

|



End-to-End Example

Suppose your XML documents are:

XML 1 (Engine)

Engine depends on Fuel System.

XML 2 (Fuel System)

Fuel System contains Pump.

XML 3 (Pump)

Pump connects to Valve.

Cell 9 Output

Subject	Predicate	Object
Engine	depends_on	Fuel System
Fuel System	contains	Pump
Pump	connected_to	Valve

↓

Cell 10

Cleans and validates these triples.

↓

Cell 11

Creates the graph:

Engine

|

depends_on

▼

Fuel System

|

contains

▼

Pump

|

connected_to

▼

Valve

Final Output of Cell 11

The final output is a **NetworkX DiGraph object named KG**.

It contains:

- **Nodes**: One node for each XML document, identified by its XML reference and storing the document title as metadata.
- **Edges**: One directed edge for each validated RDF triple, storing the semantic predicate, the supporting sentence, and the link type.

This graph is now ready for the **retrieval stage of your GraphRAG pipeline**, where queries can traverse connected XML documents and use the stored edge metadata to gather relevant context before sending it to the LLM.

Cell 16 Reteriver part

I also noticed you have **two versions**:

1. **Version 1** – Exact Entity Matching (Simple Retriever)
2. **Version 2** – Dynamic Graph Traversal (Improved Retriever)

I recommend explaining **Version 2**, because it is the improved version and more suitable for a research project.

Stage 12: GraphRAG Online Phase (Retriever)

Purpose

Until now, you have built the Knowledge Graph.

XML Files

|

Knowledge Nodes

|

Semantic Edges

|

Knowledge Graph (NetworkX)

Now the user asks a question.

Example:

How can a new Test Environment be created in the application?

The retriever must answer:

Which XML documents should I retrieve from the graph?

This is exactly what Cell 16 does.

Overall Flow

User Query

|

▼

Query Understanding

|

▼

Find Start Nodes

|

▼

Graph Traversal

|

▼

Retrieve Connected XML Documents

|

▼

Pass Context to LLM

Code Explanation

Step 1

USER_QUERY = ...

Example

How can a new Test Environment be created in the application?

This is the user's natural language question.

Nothing has been retrieved yet.

Step 2

STOPWORDS

Purpose

Remove unnecessary words.

Example

Original query

How can a new Test Environment be created in the application

Remove

how

can

a

new

be

in

the

Remaining keywords

test

environment

application

These are much better for retrieval.

Step 3

get_keywords()

Purpose

Convert text into searchable keywords.

Example

Query

How can a new Test Environment be created in the application?

↓

test

environment

application

Suppose one XML title is

Application Test Environment

↓

Keywords become

application

test

environment

Now matching becomes very easy.

Step 4

query_keywords = get_keywords(USER_QUERY)

Output

{

test,

environment,

application

}

These are the query concepts.

Step 5

Now comes the important part.

for node, attr in KG.nodes(data=True):

The retriever scans **every node in the graph**.

Suppose the graph has

Node 1

Application Test Environment

Node 2

MATLAB

Node 3

Engine Model

Node 4

Fuel System

Each node is checked.

Step 6

title_keywords

Suppose

Node

Application Test Environment

becomes

application

test

environment

Step 7

overlap =

Now compare

Query

test

environment

application

with

Node

application

test

environment

Overlap

application

test

environment

Score

3

Another node

Engine Model

Keywords

engine

model

Overlap

0

That node is ignored.

Step 8

node_scores.sort(...)

Now all nodes are ranked.

Example

Node	**Score**
Application Test Environment	3
Test Environment Configuration	2
Application Settings	1
MATLAB	0

The retriever chooses

start_nodes = node_scores[:3]

These become the graph starting points.

Graph Traversal

This is where GraphRAG differs from Vector RAG.

Suppose

Application Test Environment

is the starting node.

Graph

Application Test Environment

|

configured_by

▼

Configuration

|

contains

▼

Environment Variables

|

uses

▼

Database

Instead of retrieving only the first document,

the retriever follows the graph.

Hop 1

hop_1_nodes = explore_hop(start_nodes)

Graph

Application Test Environment

|

configured_by

▼

Configuration

Hop 1 retrieves

Configuration

Hop 2

hop_2_nodes

Now continue

Configuration

|

contains

▼

Environment Variables

Now retrieve

Environment Variables

Hop 3

Suppose only

Configuration

was found.

The retriever thinks

"I don't have enough context."

Then

if len(retrieved_edges)<5

becomes true.

Now explore another level.

Environment Variables

|

uses

▼

Database

Now Database is also retrieved.

Why Hop 3?

Because some information is hidden deeper in the graph.

Example

Question

How is MATLAB related to Turbine Engineering?

Graph

MATLAB

|

used_by

▼

Simulation

|

supports

▼

Turbine Engineering

Hop 1

Simulation

Hop 2

Turbine Engineering

Now the relationship becomes visible.

---

explore_hop()

This is the heart of the retriever.

Input

Current Nodes

Example

Configuration

The function checks

KG.successors(current_node)

Meaning

"What documents are directly connected?"

Suppose

Configuration

has

Environment Variables

Settings

Database

Then all three are retrieved.

---

visited_edges

Suppose

A

↓

B

↓

C

Later another path also reaches B.

Without

visited_edges

the retriever keeps retrieving

A→B

A→B

A→B

Again and again.

visited_edges prevents duplicate traversal.

---

retrieved_edges

Every discovered relationship is stored.

Example

Application

↓

Configuration

↓

Configuration

↓

Database

↓

Database

↓

Environment Variables

These become the context.

Final Output

Example

(Application)

|

configured_by

▼

(Configuration)

|

contains

▼

(Environment Variables)

|

uses

▼

(Database)

The retriever returns

Application

↓

Configuration

↓

Environment Variables

↓

Database

along with

their predicates

their supporting sentences

their titles

Complete Retriever Workflow

User Query

|

▼

Remove Stop Words

|

▼

Extract Query Keywords

|

▼

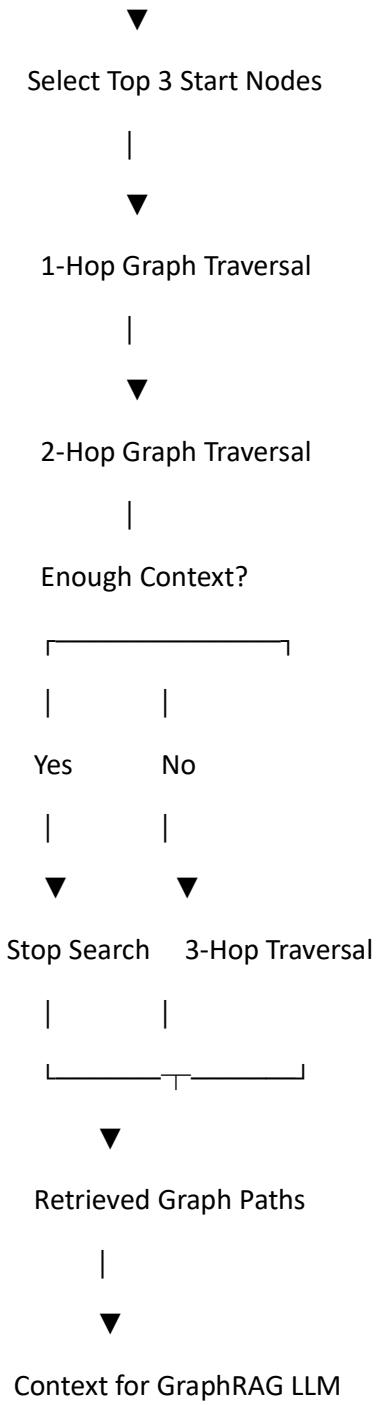
Compare with Graph Node Titles

|

▼

Rank Matching Nodes

|



Research improvement

The current retriever works, but for a stronger GraphRAG implementation, I would recommend enhancing it in later iterations:

1. **Entity extraction with an LLM or NER model** instead of keyword overlap.
2. **Rank start nodes** using similarity or PageRank, not just keyword count.

3. **Score retrieved paths** using a combination of hop distance, edge confidence, and node importance.
4. **Return the top-K most relevant paths** (for example, top 10) instead of every traversed edge before sending the context to the LLM.

These additions would make the retriever more scalable and align it more closely with research-grade GraphRAG systems.

cell 17 :-

tage 12: GraphRAG Online Phase

Cell 17: Context Builder

Summary

Purpose

The purpose of Cell 17 is to **prepare the final context for the LLM**.

Until Cell 16, we only have:

- Start Nodes
- Retrieved Graph Edges

These are useful for a machine but **not directly understandable by an LLM**.

Cell 17 converts them into readable text.

Input

From Cell 16

Start Nodes

↓

Retrieved Graph Edges

Example

Start Node

Application Test Environment

Retrieved edges

Application Test Environment

|

configured_by

▼

Configuration

Configuration

|

contains

▼

Environment Variables

Output

A final prompt like

Question

How can a Test Environment be created?

RAW DOCUMENT

<Complete XML Text>

GRAPH RELATIONSHIPS

Application Test Environment configured by Configuration.

Configuration contains Environment Variables.

This prompt is sent to the LLM.

Code Explanation

Step 1

node_texts = []

Purpose

Create an empty list to store

- XML title
- XML content

Step 2

for n in start_nodes:

Loop through every retrieved start node.

Suppose

Application Test Environment

Configuration

Each node is processed.

Step 3

title = KG.nodes[n].get(...)

Retrieve the title.

Example

Application Test Environment

Step 4

raw_content

This is important.

Instead of only retrieving

Application Test Environment

the retriever now loads the **entire XML document**.

Example

<Application Test Environment>

To create a new Test Environment

Open Settings

Click Environment

Create New Environment

Save Configuration

...

Now the LLM has the complete document.

Step 5

node_texts.append(...)

Now every document becomes

DOCUMENT TITLE

Application Test Environment

DOCUMENT CONTENT

Complete XML text...

Step 6

FULL_NODE_CONTEXT

Suppose there are three documents.

They become

DOCUMENT 1

Complete XML

DOCUMENT 2

Complete XML

DOCUMENT 3

Complete XML

Step 7

Now build graph relationships.

edge_lines=[]

Purpose

Store readable graph facts.

Step 8

for idx,(u,v,attr)

Loop over every retrieved edge.

Suppose

Application

↓

Configuration

This edge is processed.

Step 9

Retrieve titles

src_title

tgt_title

Instead of

doc:wiki:12345

we get

Application Test Environment

Much easier for the LLM.

Step 10

Cleaning IDs

if "doc:"

Suppose

the graph contains

doc:wiki:Temp_Import.3456.WebHome

The LLM cannot understand this.

So

split(".")

↓

WebHome

Remove

↓

Regex

↓

Keep only words

Result

Temp Import

or

External Document

This greatly improves prompt readability.

Step 11

Clean predicate

Current

depends_on

becomes

depends on

Current

created_by

becomes

created by

Now the prompt reads naturally.

Step 12

Build graph facts

Suppose

Application

↓

configured_by

↓

Configuration

Becomes

Application configured by Configuration.

Another

Configuration

↓

contains

↓

Environment Variables

becomes

Configuration contains Environment Variables.

Step 13

Join all relationships

GRAPH_EDGE_CONTEXT

Result

1.

Application configured by Configuration.

2.

Configuration contains Environment Variables.

3.

Environment Variables use Database.

Now the graph is human-readable.

Step 14

System Prompt

This is extremely important.

The system prompt tells the LLM how to behave.

Rules

1.

Only use Graph Context

↓

No hallucination

2.

If graph is empty

↓

Say

Information unavailable.

3.

Otherwise

↓

Analyze deeply

Don't simply copy sentences.

Explain relationships.

This is exactly what GraphRAG should do.

Step 15

FINAL_PROMPT

Now everything is merged.

Example

Question

How can a new Test Environment be created?

RAW DOCUMENT TEXT

Application Test Environment

Complete XML Text

Configuration

Complete XML Text

GRAPH RELATIONSHIPS

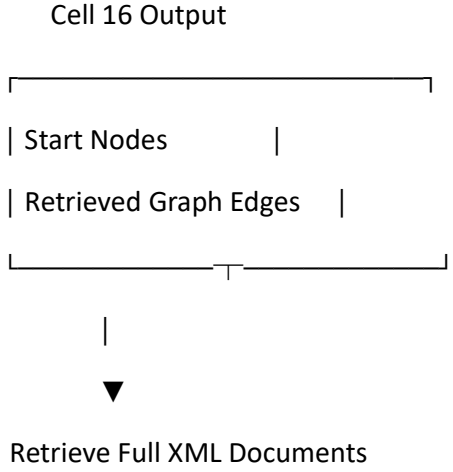
Application configured by Configuration.

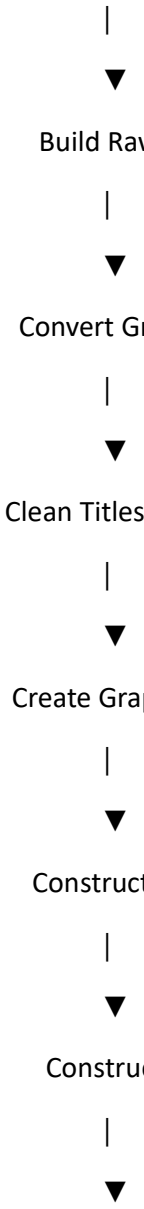
Configuration contains Environment Variables.

Environment Variables use Database.

This becomes the LLM input.

Overall Workflow





Example End-to-End

User Query

How can a new Test Environment be created?

Retrieved Start Node

Application Test Environment

Raw XML

Application Test Environment

To create a new Test Environment:

1. Open Settings.

2. Click Environment.

3. Select Create New.

4. Save Configuration.

Retrieved Graph

Application Test Environment

|

configured_by

▼

Configuration

Configuration

|

contains

▼

Environment Variables

Final Prompt Sent to LLM

Question:

How can a new Test Environment be created?

RAW DOCUMENT TEXT

Application Test Environment

To create a new Test Environment:

1. Open Settings

2. Click Environment

3. Create New

4. Save Configuration

GRAPH RELATIONSHIPS

Application Test Environment configured by Configuration.

Configuration contains Environment Variables.

Final Output of Cell 17

The final output of Cell 17 is **FINAL_PROMPT**, which combines:

- The **user's question**.
- The **full text of the retrieved XML documents** (FULL_NODE_CONTEXT).
- The **semantic graph relationships** extracted during graph traversal (GRAPH_EDGE_CONTEXT).
- A **system prompt** that instructs the LLM to answer **only from the retrieved Knowledge Graph context** and to synthesize information from both the raw document text and the graph relationships.

This FINAL_PROMPT is then passed to **Cell 18**, where the LLM generates the final GraphRAG answer

Stage 12: GraphRAG Online Phase

Cell 18: Generate Answer using LLM

Summary

Purpose

The purpose of Cell 18 is to **send the retrieved Knowledge Graph context to the LLM and generate the final answer for the user**.

Until Cell 17, we have already built the complete prompt.

Now this cell simply asks the LLM:

"Here is the user's question. Here is the retrieved graph context. Generate the answer."

Input

From Cell 17

SYSTEM_PROMPT

+

FINAL_PROMPT

Example

Question

How can a new Test Environment be created?

RAW DOCUMENT

Application Test Environment

...

GRAPH RELATIONSHIPS

Application configured by Configuration.

Configuration contains Environment Variables.

Output

Example

To create a new Test Environment:

1. Open the Application Settings.

2. Navigate to Environment Configuration.

3. Select Create New Environment.

4. Configure the required Environment Variables.

5. Save the configuration.

The Knowledge Graph indicates that the Test Environment is configured through the Configuration module, which contains the required Environment Variables.

This is the final GraphRAG answer presented to the user.

Code Explanation

Step 1

print(...)

Purpose

Display

GRAPHRAG ONLINE: FINAL LLM ANSWER

Only for logging.

Step 2

try:

Purpose

If the Azure OpenAI request fails,

the notebook will not crash.

Instead,

the error will be displayed.

Step 3

response = llm.invoke(...)

This is the most important line.

Here the LLM is called.

You are using

Azure OpenAI

↓

LangChain

↓

llm.invoke()

What is sent to the LLM?

[

("system", SYSTEM_PROMPT),

("human", FINAL_PROMPT)

]

There are two messages.

System Message

SYSTEM_PROMPT

This tells the LLM how it should behave.

Example

Only answer using the retrieved Knowledge Graph.

Do not use outside knowledge.

If no context exists,

reply

"The information is unavailable in the Knowledge Graph."

Think of it as the **instructions** for the model.

Human Message

FINAL_PROMPT

Contains

Question

RAW DOCUMENT TEXT

GRAPH RELATIONSHIPS

Example

Question

How can a Test Environment be created?

RAW DOCUMENT

Complete XML

GRAPH

Application configured by Configuration.

Configuration contains Environment Variables.

This is the **actual information** the model uses to answer the user's question.

Step 4

What happens inside the LLM?

Conceptually, the process is:

User Question

|

▼

System Prompt

|

▼

Raw XML Context

|

▼

Graph Relationships

|

▼

Reasoning

|

▼

Final Answer

The LLM combines:

- User question
- Raw document text
- Semantic graph relationships

to produce a grounded answer.

Step 5

response.content.strip()

The Azure OpenAI response contains additional metadata.

For example:

Response

content

metadata

usage

finish_reason

You only need the generated text.

So

response.content

extracts

The Test Environment can be created...

Why .strip()?

Suppose the LLM returns

The Test Environment can be created...

with blank lines.

.strip() removes unnecessary whitespace.

Step 6

final_answer

Now the answer is stored.

Example

The Test Environment is created by navigating to the Configuration module and selecting Create Environment.

The graph indicates that the Configuration module contains the required Environment Variables.

Therefore these must be configured before saving the new environment.

Step 7

Print the result

print(USER_QUERY)

Displays

USER QUESTION

How can a new Test Environment be created?

Then

print(final_answer)

Displays

GRAPHRAG ANSWER

...

This is the answer shown to the user.

Step 8

except Exception

Suppose Azure OpenAI is unavailable.

Instead of

Notebook Failed

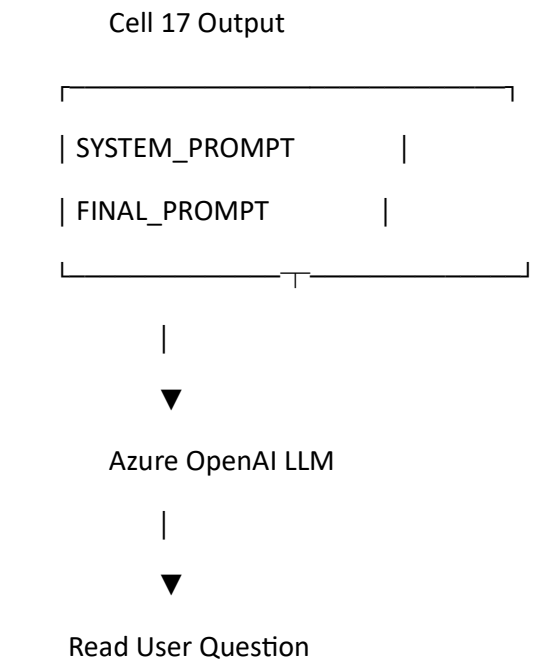
the notebook displays

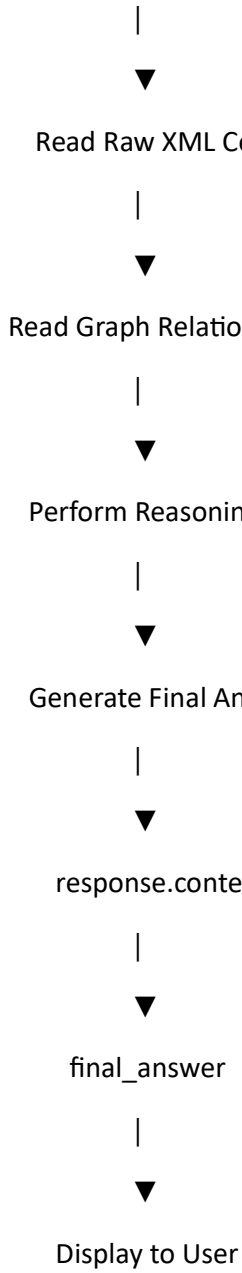
Error calling LLM

Connection Timeout

This makes debugging much easier.

Complete Workflow





End-to-End Example

User Question

How can a new Test Environment be created?

Context from Cell 17

Raw XML

Application Test Environment

To create a new Test Environment:

Open Settings

Create Environment

Save Configuration

Graph Relationships

Application configured by Configuration.

Configuration contains Environment Variables.

LLM Input

System Prompt

↓

Question

↓

Raw XML

↓

Graph Relationships

LLM Output

To create a new Test Environment:

1. Open the Application Settings.

2. Navigate to the Configuration module.

3. Select Create Environment.

4. Configure the required Environment Variables.

5. Save the configuration.

According to the Knowledge Graph, the Test Environment is configured through the Configuration module, which contains the required Environment Variables.

Overall GraphRAG Online Pipeline

User Question

|

▼

Cell 16

Query Understanding

|

▼

Graph Traversal

|

▼

Retrieved Nodes & Edges

|

▼

Cell 17

Context Builder

|

▼

SYSTEM_PROMPT

+

FINAL_PROMPT

|

▼

Cell 18

Azure OpenAI (LLM)

|

▼

Reasoning over

• Raw XML Content

• Graph Relationships

|

▼

Final GraphRAG Answer

|

▼

Displayed to the User

Final Output of Cell 18

The final output of Cell 18 is the **generated answer (final_answer)** returned by the Azure OpenAI LLM.

The model generates this answer by combining:

- The **user's question**.

- The **retrieved XML document content**.
- The **semantic relationships retrieved from the Knowledge Graph**.
- The **rules defined in the system prompt** (for example, answering only from the retrieved graph context).

This is the final stage of the GraphRAG pipeline, where all the work done in the previous cells is used to produce a grounded, context-aware response for the user.

Offline Phase

XML

↓

Cleaning

↓

Knowledge Nodes

↓

Hyperlink Extraction

↓

Sentence Extraction

↓

LLM Predicate Extraction

↓

RDF Triples

↓

Knowledge Graph

Online Phase

User Query

↓

Query Understanding

↓

Graph Traversal

↓

Context Builder

↓

LLM

↓

Answer